Final Year Project

HALITE

AI-Powered Maintenance Diagnostics and Asset Intelligence Platform

FYP Team

Shahmeer Atif (23I-0711)

Muhammad Umar (23I-0782)

Muhammad Usman Al Haq (23I-0585)

Supervised by

Syed Muhammad Saad Salman

FAST School of Computing

National University of Computer and Emerging Sciences

Islamabad Campus, Pakistan

October 2026

Students' Submission

This report, titled **Halite: AI-Powered Maintenance Diagnostics and Asset Intelligence Platform**, is submitted as the Final Year Project (FYP-I / FYP-II) report of the BS programme, session 2023–2027, at the FAST School of Computing, National University of Computer and Emerging Sciences, Islamabad Campus.

Anti-Plagiarism Declaration

This is to declare that the above FYP report, produced under the title "Halite: AI-Powered Maintenance Diagnostics and Asset Intelligence Platform", is the sole contribution of the author(s) and no part hereof has been reproduced on an as-it-is basis (cut and paste) which can be considered as plagiarism. All referenced parts have been used to argue the idea and have been cited properly. We will be responsible and liable for any consequence if a violation of this declaration is determined.

Student 1
Name: Shahmeer Atif
Signature: _____

Student 2
Name: Muhammad Umar
Signature: _____

Student 3
Name: Muhammad Usman Al Haq
Signature: _____

Supervisor (Faculty)
Name: Syed Muhammad Saad Salman
Signature: _____

Date: _____

Authors' Declaration

The authors declare that the work presented in this report is their own and has not been submitted or presented previously to any other institution or organisation.

Abstract

Maintenance organisations hold large archives of service reports whose diagnostic value is lost because records are fragmented, unstructured and sometimes superseded by later findings. Halite is an AI-powered maintenance diagnostics and asset intelligence platform that converts historical service records into an evidence-aware knowledge system. It combines a maintenance knowledge graph, semantic retrieval, correction-aware fault ranking, conversational diagnosis, conflict detection with human review, technician training and fleet analytics. Its central contribution is a diagnostic pipeline that retrieves relevant cases, ranks investigation hypotheses, verifies citations and communicates uncertainty. The system will be built as a secure multi-user SaaS application and evaluated against keyword, semantic and direct-LLM baselines using FAA Service Difficulty Report data.

Keywords: Maintenance Intelligence, Knowledge Graph, Fault Diagnosis, Explainable AI, Graph Retrieval, Retrieval-Augmented Generation, Conflict Detection, Technician Training, SaaS.

Executive Summary

The Problem

Maintenance knowledge is spread across historical records that may be incomplete, hard to search, contradictory or outdated. A technician investigating a recurring fault must work out which symptoms have occurred before, which components were investigated, which interventions failed, what was ultimately found, which records remain reliable and what to investigate next. This is a problem of reasoning over evolving technical evidence, not merely of document search.

The Solution

Halite is a maintenance-focused SaaS platform. It identifies relevant evidence, distinguishes earlier repair attempts from later findings, ranks possible investigation paths and explains which records support each response. Its nine modules are:

- Interactive AI diagnostic assistant.
- Asset health and maintenance history.
- Maintenance knowledge graph.
- Conflict and misdiagnosis intelligence.
- Maintenance report ingestion.
- Technician training simulator.
- Fleet-level maintenance analytics.
- Collaborative supervisor workflows.
- Organisation and user administration.

Research Contribution

The academically significant work lies in graph-constrained candidate retrieval, evidence-aware diagnostic ranking, correction-aware historical memory, confidence and abstention policies, and reproducible diagnostic evaluation. These contributions are evaluated against keyword, semantic, graph-only, hybrid and direct language-model baselines on FAA Service Difficulty Report data (about 2,079 records covering roughly 60 airframes).

Development Plan

The team will establish the core diagnostic system in FYP-I. Advanced diagnostic workflows, training simulation, analytics and production hardening follow in FYP-II, with an internal target of approximately 90 to 95 percent of the planned functional scope by FYP-II Mid and the Job Fair.

Expected Outcomes

The project will deliver a deployed, tested and documented multi-user SaaS system with a maintenance knowledge graph, an evidence-grounded diagnostic engine, a conflict-aware record

workflow, a technician training module, maintenance analytics, automated test suites and reproducible evaluation results.

Project Team

The project is carried out by Shahmeer Atif (diagnostic interaction, frontend integration and product UX), Muhammad Umar (knowledge graph, retrieval, ranking and evaluation) and Muhammad Usman Al Haq (data processing, conflict intelligence, CI/CD and deployment), under the supervision of Syed Muhammad Saad Salman.

Safety Boundary

Halite is a decision-support system. It does not approve repairs, certify aircraft or replace licensed inspectors, and it always states its uncertainty.

Table of Contents

1 Introduction 1

1.1 Overview 1

1.2 Background and Motivation 1

1.3 Organisation of This Report 1

2 Research on Existing Ideas and Products 3

2.1 Research Methodology 3

2.2 Computerized Maintenance Management Systems 3

2.3 Keyword-Based Retrieval 3

2.4 Semantic Retrieval 3

2.5 Retrieval-Augmented Generation 4

2.6 Knowledge Graph Retrieval 4

2.7 Case-Based Reasoning 4

2.8 Conflict Detection 4

2.9 Research Gap 4

2.10 Comparison of Approaches 5

3 Project Vision 6

3.1 Problem Statement 6

3.2 Vision Statement 6

3.3 Business Opportunity 6

3.4 Objectives 6

3.5 Project Scope 7

3.6 Constraints 7

3.7 Stakeholders 8

3.8 Sustainable Development Goal Alignment 8

3.9 DEI, Ethical and Legal Statements 9

4 Complex Computing Problem and Research Questions 10

4.1 Complex Computing Problem 10

4.2 Sources of Complexity 10

4.3 Research Questions 10

4.4 Academic Contribution 10

4.5 Student Ownership 11

5 Product Specification 12

5.1 System Modules 12

5.2 M01: Workspace and Identity 12

5.3 M02: Asset Intelligence 13

5.4 M03: Maintenance Knowledge Graph 13

5.5 M04: AI Diagnostic Workspace 14

5.6 M05: Conflict Intelligence 15

5.7 M06: Maintenance Report Intelligence 16

5.8 M07: Technician Academy 17

5.9 M08: Maintenance Analytics 17

5.10 M09: Administration and Governance....18
6 Software Requirements Specification....19
6.1 Requirement Identification....19
6.2 List of Features....19
6.3 Functional Requirements....19
6.4 Non-Functional Requirements....20
7 Use Cases and User Interface....21
7.1 Use Case Overview....21
7.2 UC-05: Start Diagnostic Investigation....22
7.3 UC-08: Review Historical Conflict....22
7.4 UC-10: Complete Training Scenario....23
7.5 Wireframes....23
8 System Architecture....25
8.1 Architectural Style....25
8.2 Frontend....25
8.3 Backend....26
8.4 Graph Database....26
8.5 Relational Database....26
8.6 External Language Models....26
8.7 Deployment Architecture....26
9 Data and Knowledge Graph Design....27
9.1 Primary Data Source....27
9.2 Evaluation Corpus....27
9.3 Graph Node Types....27
9.4 Graph Edge Types....28
9.5 Graph Integrity Rules....29
9.6 Fault Case Model....29
9.7 Entity Resolution....30
9.8 Data Provenance....30
10 Diagnostic Intelligence Algorithms....31
10.1 Diagnostic Processing Pipeline....31
10.2 Symptom Extraction....31
10.3 Graph-Constrained Retrieval....31
10.4 Semantic Retrieval....32
10.5 Hybrid Candidate Retrieval....32
10.6 Historical Correction Penalty....32
10.7 Fault Hypothesis Construction....32
10.8 Confidence and Abstention....33
10.9 Question Selection....33
10.10 Evidence Verification....33
10.11 Algorithm Pseudocode....34
10.12 Conflict Classification Engine....34
10.13 Diagnostic Evaluation Baselines....34
11 Evaluation and Research Methodology....35

11.1 Research Design....35
11.2 Dataset Preparation....35
11.3 Avoiding Data Leakage....35
11.4 Diagnostic Ground Truth....35
11.5 Retrieval Metrics....35
11.6 Diagnostic Ranking Metrics....36
11.7 Confidence Evaluation....36
11.8 Citation Evaluation....36
11.9 Conflict Evaluation....36
11.10 Ablation Study....36
11.11 User Acceptance Testing....36
11.12 Training Evaluation....37

12 Security and Safety....38
12.1 Security Requirements....38
12.2 Multi-Tenant Isolation....38
12.3 LLM Security....38
12.4 Operational Safety....38

13 Iteration Plan....39
13.1 Development Approach....39
13.2 Proposal Defense....39
13.3 Iteration 1: FYP-I Mid Evaluation....40
13.4 Iteration 2: FYP-I Final Evaluation....40
13.5 Iteration 3: FYP-II Early Development....41
13.6 Iteration 4: FYP-II Mid and Job Fair....41
13.7 Iteration 5: FYP-II Final....42

14 Team Responsibilities....43
14.1 Team Composition....43
14.2 Shahmeer Atif....43
14.3 Muhammad Umar....43
14.4 Muhammad Usman Al Haq....43
14.5 Shared Responsibilities....44
14.6 Individual Ownership Evidence....44

15 CI/CD and Software Engineering Practices....45
15.1 Version Control....45
15.2 Branch Conventions....45
15.3 Continuous Integration....45
15.4 Continuous Deployment....45
15.5 Repository Structure....46
15.6 Continuous Integration Evidence....46
15.7 Definition of Done....46

16 Testing Strategy....47
16.1 Testing Objectives....47
16.2 Unit Testing....47
16.3 Integration Testing....47

16.4 System Testing..... 47
16.5 Security Testing..... 47
16.6 Performance Testing..... 47
16.7 Failure-Mode Testing..... 48
16.8 Acceptance Testing..... 48
17 Project Risks and Mitigation..... 49
18 Expected Results and Deliverables..... 50
18.1 Expected Technical Results..... 50
18.2 Expected Research Results..... 50
18.3 Documentation Deliverables..... 50
18.4 Commercial Demonstration..... 50
19 Conclusion..... 51
References..... 52
Appendix A: Mid Evaluation Readiness Checklist..... 53
A.1 Live Core Module..... 53
A.2 CI/CD..... 53
A.3 Documentation..... 53
Appendix B: Diagnostic Acceptance Cases..... 54
B.1 Case A: Known Historical Correction..... 54
B.2 Case B: No Relevant History..... 54
B.3 Case C: Conflicting Evidence..... 54
B.4 Case D: Cross-Tenant Access..... 54
Appendix C: Research Experiment Matrix..... 55
Appendix D: Final Deliverable Checklist..... 56

List of Figures

Figure 7.1: Primary Use Case Diagram..... 21
Figure 7.2: Wireframe of the Diagnostic Workspace..... 23
Figure 7.3: Wireframe of the Asset Detail Page..... 24
Figure 8.1: High-Level System Architecture..... 25
Figure 9.1: Knowledge Graph Schema..... 29
Figure 10.1: Diagnostic Processing Pipeline..... 31
Figure 13.1: Iteration Roadmap..... 39

List of Tables

Table 2.1: Architectural Comparison of Retrieval Approaches....5
Table 3.1: Stakeholders and Their Goals....8
Table 4.1: Sources of Complexity....10
Table 5.1: Halite Product Modules....12
Table 6.1: Functional Requirements....19
Table 6.2: Non-Functional Requirements....20
Table 7.1: Primary Use Cases....21
Table 9.1: Planned Evaluation Corpus....27
Table 9.2: Knowledge Graph Schema: Node Types....27
Table 9.3: Knowledge Graph Schema: Relationship Types....28
Table 13.1: Summary of Evaluation Stages....39
Table 14.1: Team Ownership....43
Table 17.1: Project Risk Register....49
Table C.1: Research Experiment Matrix....55
Table D.1: Final Deliverable Checklist....56

Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CCP	Complex Computing Problem
CI/CD	Continuous Integration / Continuous Deployment
CMMS	Computerised Maintenance Management System
CSV	Comma-Separated Values
DEI	Diversity, Equity and Inclusion
FAA	Federal Aviation Administration
FYP	Final Year Project
JASC	Joint Aircraft System/Component Code
LLM	Large Language Model
MRR	Mean Reciprocal Rank
MVP	Minimum Viable Product
NFR	Non-Functional Requirement
OCR	Optical Character Recognition
PoC	Proof of Concept
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
SaaS	Software as a Service
SDG	Sustainable Development Goal
SDR	Service Difficulty Report
SDS	Software Design Specification
SRS	Software Requirements Specification
SUS	System Usability Scale
UAT	User Acceptance Testing
UX	User Experience

1 Introduction

1.1 Overview

Halite is an AI-powered maintenance diagnostics and asset intelligence platform that helps maintenance teams interpret and reuse the technical knowledge stored in historical service records. Maintenance work frequently involves reasoning from observed symptoms to possible causes by drawing on earlier observations, inspections and repairs. A technician may recognise an initial symptom but not know about less obvious component failures that earlier reports have already documented, and a useful observation can become hard to find once hundreds of reports have accumulated.

Halite addresses this by organising maintenance knowledge around physical assets, reported symptoms, associated components, inspection outcomes and repairs. The platform will retrieve relevant historical cases, identify which findings were later corrected, rank investigation hypotheses and explain every recommendation with references to the records that support it.

1.2 Background and Motivation

Maintenance organisations accumulate service reports, inspection records, corrective actions and technician notes over the operational life of each asset. These records hold valuable diagnostic knowledge, but they are fragmented, unstructured and frequently inconsistent. Conventional maintenance software stores records, administers work orders and schedules servicing. Even when records are searchable, a retrieved report may describe an unsuccessful intervention, an outdated diagnosis or a finding that another technician later corrected.

A technician therefore needs more than a list of similar reports. The technician needs to know which symptoms have occurred before, which components were investigated, which interventions were attempted and failed, what was ultimately found, which records remain reliable and what should be investigated next. For example, if an earlier repair was followed by a report that identified a different component, the later evidence should shape every future investigation of the same symptom.

This is not merely a document-search problem. It is a problem of reasoning over evolving technical evidence, and it motivates a platform that combines structured retrieval, correction awareness and explainable AI.

1.3 Organisation of This Report

Chapter 2 reviews existing approaches and identifies the research gap. Chapter 3 defines the project vision, scope, constraints, stakeholders and ethical considerations. Chapter 4 states the

complex computing problem and research questions. Chapters 5 to 7 specify the product, its requirements and its use cases. Chapters 8 to 10 describe the architecture, data model and diagnostic algorithms. Chapters 11 and 12 present the evaluation methodology and security design. Chapters 13 to 18 cover the iteration plan, team responsibilities, engineering practices, testing, risks and expected deliverables. Chapter 19 concludes the report.

2 Research on Existing Ideas and Products

2.1 Research Methodology

The project combines application-oriented research with experimental evaluation. The research work consists of the following activities:

1. Analysis of maintenance workflows.
2. Review of relevant software categories.
3. Study of knowledge graph retrieval.
4. Study of AI-assisted fault diagnosis.
5. Examination of historical maintenance data.
6. Development of a working system.
7. Controlled evaluation against baselines.
8. User acceptance and usability testing.

2.2 Computerized Maintenance Management Systems

Traditional CMMS software is primarily associated with asset administration, work-order management, preventive maintenance, maintenance history, inspection schedules and service inventory. Halite does not attempt to reproduce the entire CMMS product category. Its differentiating focus is the interpretation of historical maintenance evidence. This report makes no claim about the capabilities of specific commercial vendors; a formal product comparison will be completed before any such claim is made.

2.3 Keyword-Based Retrieval

Keyword search returns documents that match literal terms. The approach is inexpensive and interpretable, but it can miss semantically equivalent descriptions. For example, two technicians may describe the same warning condition using different abbreviations, and a keyword query will retrieve only one of the two reports.

2.4 Semantic Retrieval

Embedding-based retrieval maps text into numerical vector representations, so similar records can be identified even when the exact words differ. However, semantic similarity does not indicate whether a historical intervention was successful or has since been superseded.

2.5 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines retrieved evidence with language generation (Lewis et al., 2020), allowing a large language model to produce contextual responses grounded in a document collection. Self-reflective variants additionally learn to critique retrieved passages (Asai et al., 2023). A standard RAG implementation can still retrieve outdated or contradictory records and present them as current. Halite therefore adds an evidence-interpretation layer between retrieval and generation.

2.6 Knowledge Graph Retrieval

Knowledge graphs represent relationships among entities. A maintenance graph can explicitly connect assets and records, symptoms and fault cases, components and inspections, findings and repairs, and reports and their corrections. GraphRAG research illustrates how structured relationships can improve retrieval workflows (Edge et al., 2024). Halite adapts this principle to maintenance-specific data and investigation paths.

2.7 Case-Based Reasoning

Case-based reasoning solves new problems by reference to historical examples. A new symptom can be compared with earlier cases that involve similar assets, components or observed behaviour. Halite extends case retrieval with the following capabilities:

1. Record-validity awareness.
2. Graph relationships.
3. Candidate ranking.
4. Evidence provenance.
5. Uncertainty control.

2.8 Conflict Detection

Historical corrections carry information about incorrect or incomplete earlier findings. A conflict-detection engine can classify the relationship between a new record and an earlier one as contradicts, supersedes, partially supersedes, scope disjoint or no conflict. The diagnostic engine uses these relationships to avoid uncritically recommending interventions that history has shown to be unsuccessful.

2.9 Research Gap

The research gap addressed by Halite is the integration and evaluation of maintenance diagnostic retrieval with explicit evidence validity. A diagnostic system must answer not only the question

"which historical records are similar to this symptom?" but also "which historical findings remain supported, which were corrected, and what investigation hypotheses do the combined records justify?"

2.10 Comparison of Approaches

Table 2.1 compares the three retrieval approaches at an architectural level. The comparison describes design differences and does not establish experimental superiority; that question is addressed by the evaluation in Chapter 11.

Table 2.1: Architectural Comparison of Retrieval Approaches

Capability	Keyword	Semantic RAG	Halite
Natural-language input	Limited	Yes	Yes
Asset-specific context	Manual	Filtered	Graph-aware
Component relations	No	Implicit	Explicit
Historical corrections	No	Not intrinsic	Explicit
Fault hypotheses	No	Generated	Ranked
Evidence checking	Manual	Partial	Deterministic
Uncertainty handling	No	Model-based	Policy-based
Training simulation	No	Optional	Dedicated

3 Project Vision

3.1 Problem Statement

Given an identified maintenance asset, a natural-language symptom description, a corpus of historical service reports, structured asset and component information, and potentially contradictory historical findings, the project will design a system that retrieves relevant evidence, constructs diagnostic hypotheses, ranks investigative possibilities and explains its recommendations without treating unsupported model output as verified fact. The system must preserve provenance, identify conflicting historical claims and allow human intervention.

3.2 Vision Statement

> To transform fragmented maintenance records into an accessible and explainable diagnostic intelligence platform that assists maintenance teams in investigating failures, understanding prior interventions and learning from historical cases.

3.3 Business Opportunity

Halite can serve maintenance organisations that hold extensive historical records but have limited diagnostic search and learning capabilities. The anticipated value includes:

- Better reuse of historical knowledge.
- Reduced manual record inspection.
- Easier discovery of corrective findings.
- More consistent diagnostic evidence.
- Improved technician onboarding.
- Better visibility into recurring faults.

These are anticipated benefits. Actual time savings and organisational improvements will be validated through controlled experiments and stakeholder testing rather than assumed.

3.4 Objectives

3.4.1 Primary Objective

To design, implement and evaluate a knowledge-graph-grounded, AI-assisted maintenance diagnostics SaaS platform that improves historical evidence retrieval and supports explainable fault investigation.

3.4.2 Specific Objectives

1. Develop a structured maintenance knowledge graph.
2. Implement maintenance-record ingestion and normalisation.
3. Build asset-aware historical retrieval.
4. Implement semantic retrieval for technical symptoms.
5. Develop an evidence-aware diagnostic ranking algorithm.
6. Provide interactive symptom investigation through conversational AI.
7. Detect superseded, contradictory and unresolved maintenance findings.
8. Implement human review for uncertain record decisions.
9. Generate educational diagnostic scenarios from historical cases.
10. Provide recurring-fault and maintenance analytics.
11. Build a secure multi-user SaaS application.
12. Evaluate the algorithms against controlled baselines.
13. Deploy the system with automated testing and CI/CD.

3.5 Project Scope

3.5.1 In Scope

Halite is a complete maintenance intelligence application comprising nine modules: workspace and identity, asset intelligence, maintenance knowledge graph, AI diagnostic workspace, conflict intelligence, maintenance report intelligence, technician academy, maintenance analytics and administration. The core evaluation context is historical aviation maintenance data from the FAA Service Difficulty Report (SDR) collection.

3.5.2 Boundaries

- The software architecture uses a configurable asset model, but performance claims for other industries are excluded unless additional datasets are evaluated.
- Parts-demand forecasting is excluded, because SDR data records reported difficulties rather than complete parts consumption.
- The platform is a decision-support system. It does not approve repairs, certify airworthiness or replace licensed inspectors.

3.6 Constraints

- **Academic timeline:** The system must be designed, built, evaluated and defended within FYP-I and FYP-II by a team of three students.
- **Data availability:** The evaluation relies on public FAA SDR data, which does not confirm the root cause of every reported fault.

- **Access to practitioners:** Qualified maintenance personnel may not be available for user testing, in which case proxy participants will be declared.
- **External services:** Language-model and embedding services introduce cost, rate-limit and availability constraints.
- **Resources:** Hosting, graph database and storage resources must fit within student-project budgets.
- **Safety boundary:** Outputs must always be framed as investigative support subject to approved maintenance documentation.

3.7 Stakeholders

3.7.1 Stakeholder Summary

Table 3.1: Stakeholders and Their Goals

Stakeholder	Role in the System	Key Goals and Problems
Maintenance Technician	Describes symptoms, inspects asset history and investigates potential causes.	Quickly find relevant past cases; avoid repeating failed repairs.
Maintenance Supervisor	Reviews uncertain findings and monitors maintenance activity.	Trust decisions; see the evidence behind every conflict.
Training Instructor	Uses historical cases to teach diagnostic reasoning.	Realistic scenarios with traceable source cases.
Organisation Administrator	Manages membership, roles, data access and configuration.	Secure, auditable control of tenant data.
Quality Reviewer	Examines recurring faults and inconsistent records.	Visibility of recurring faults and unsuccessful interventions.

3.7.2 Key Goals and Problems

The common problem across stakeholders is that useful evidence exists in historical records but cannot be found, trusted or interpreted quickly. The shared goal is a system that surfaces relevant evidence, shows which findings were corrected, and states clearly when the evidence is insufficient.

3.8 Sustainable Development Goal Alignment

Halite aligns with SDG 9, Industry, Innovation and Infrastructure, through its focus on digital maintenance knowledge and technical infrastructure. The SDG designation is not presented as evidence of computing novelty; the technical contribution is defined in Chapter 4.

3.9 DEI, Ethical and Legal Statements

3.9.1 Diversity, Equity and Inclusion

- Role-based access ensures that every user type receives an interface suited to its tasks, without assuming prior expertise in graph or AI concepts.
- Training analytics will support learning and will not be used as punitive performance surveillance of individual technicians.
- Interface text will use plain, unambiguous language and standard accessibility practices such as sufficient contrast and keyboard navigation.

3.9.2 Ethical and Safety Statement

Halite is an investigative decision-support system. It must not approve aircraft repairs, certify airworthiness, replace licensed inspectors, authorise maintenance release or present uncertain hypotheses as verified diagnoses. Any physical inspection guidance must be interpreted through approved maintenance documentation and qualified personnel.

3.9.3 Legal and Data Governance

The team will record the source, acquisition date, licence terms, permitted uses, restrictions, preprocessing steps and version of every dataset used. Each organisation's data remains isolated from every other organisation.

3.9.4 AI-Assisted Development

AI tools may support code development, documentation, testing and research, subject to university policy. The team remains responsible for system design, algorithm selection, technical reasoning, validation, error analysis and academic defence. An AI usage log will identify significant tool assistance and the verification performed by the responsible team member.

4 Complex Computing Problem and Research Questions

4.1 Complex Computing Problem

> How can a diagnostic system efficiently identify, rank and explain plausible fault hypotheses from heterogeneous, temporally evolving and potentially contradictory maintenance records, while preserving evidence provenance and avoiding unsupported high-confidence conclusions?

4.2 Sources of Complexity

Table 4.1: Sources of Complexity

Source	Description
Heterogeneous data	Maintenance records contain free text, identifiers, component codes, dates and inconsistent terminology.
Incomplete information	Many records lack complete fault descriptions, inspection sequences or final outcomes.
Conflicting evidence	Different reports may attribute the same symptom to different components.
Temporal evolution	A diagnosis may change after additional inspection, so the historical sequence must be represented.
Large candidate space	Comparing every record with every other record creates unnecessary computational work.
Conflicting objectives	Diagnostic recall and false-positive control must be balanced.
Evidence accountability	Each surfaced conclusion needs traceable supporting information.

4.3 Research Questions

RQ1: Does graph-constrained retrieval improve relevant case discovery compared with keyword retrieval?

RQ2: Does hybrid graph and semantic retrieval improve fault-evidence ranking over semantic retrieval alone?

RQ3: Does correction-aware retrieval reduce misleading historical recommendations?

RQ4: Can uncertainty-aware abstention reduce unsupported diagnostic suggestions?

RQ5: Can case-based interactive training improve learner performance on held-out diagnostic scenarios?

4.4 Academic Contribution

The team's contribution is organised into four areas.

4.4.1 Graph-Constrained Retrieval

Candidate evidence is selected using explicit relationships among assets, symptoms, components and repairs.

4.4.2 Diagnostic Ranking

A ranking mechanism combines structured evidence signals into a score for each fault hypothesis.

4.4.3 Correction-Aware Memory

Superseded records and historical contradictions are represented explicitly, so earlier unsuccessful interventions remain visible as history without being treated as current guidance.

4.4.4 Confidence-Aware Decisions

The system may abstain, request more symptoms or recommend human review when evidence is weak.

4.5 Student Ownership

The team owns the problem formulation, algorithm design, evaluation criteria and failure analysis. External language-model services, database engines and frameworks are supporting components. The project contribution remains identifiable if the external language model is replaced.

5 Product Specification

5.1 System Modules

Halite consists of nine modules, summarised in Table 5.1 and described in the sections that follow.

Table 5.1: Halite Product Modules

ID	Module	Purpose
M01	Workspace	Accounts, organisations, roles and access.
M02	Assets	Asset profiles and maintenance timelines.
M03	Knowledge Graph	Structured maintenance relationships.
M04	Diagnostics	AI-assisted symptom investigation.
M05	Conflict Intelligence	Record contradictions and corrections.
M06	Report Intelligence	Maintenance record ingestion and extraction.
M07	Technician Academy	Training and diagnostic simulation.
M08	Analytics	Fleet and maintenance insights.
M09	Administration	Audit, configuration and governance.

5.2 M01: Workspace and Identity

5.2.1 Purpose

This module provides the organisational foundation for SaaS operation.

5.2.2 Major Features

1. Registration and login.
2. Organisation creation.
3. Member invitations.
4. Role-based permissions.
5. Profile management and password reset.
6. Workspace settings.
7. Account deactivation.
8. Session management.
9. Tenant-scoped data access.

5.2.3 Roles

- **Technician:** performs investigations and submits records.
- **Supervisor:** reviews uncertain cases and maintenance intelligence.
- **Instructor:** manages training material.

- **Administrator:** manages users, assets and organisational settings.

5.2.4 Acceptance Criterion

A technician belonging to Organisation A must not be able to access the assets or records of Organisation B.

5.3 M02: Asset Intelligence

5.3.1 Purpose

Each physical asset receives a consolidated profile.

5.3.2 Major Features

1. Asset registry with search and filters.
2. Component inventory.
3. Chronological record timeline.
4. Fault history and previous repairs.
5. Recurring symptoms.
6. Relevant diagnostic sessions.
7. Linked documents.
8. Asset-specific graph view.

5.3.3 Asset Detail Page

The asset detail page will contain the asset identity and metadata, maintenance history, reported fault categories, recurring symptoms, relevant component relationships, active review alerts and the diagnostic investigation history.

5.3.4 Acceptance Criterion

Selecting an asset must scope historical retrieval to that asset unless cross-asset fleet evidence is explicitly requested.

5.4 M03: Maintenance Knowledge Graph

5.4.1 Purpose

The graph stores maintenance relationships in an explainable structure.

5.4.2 Major Features

1. Asset-to-record and record-to-symptom relationships.
2. Symptom-to-component associations.

3. Component-to-finding relationships.
4. Repair outcome relationships.
5. Record chronology and supersession relationships.
6. Graph traversal.
7. Subgraph visualisation.
8. Evidence-path exploration.

5.4.3 Example Evidence Path

A typical path runs from an asset to a reported symptom, from the symptom to a historical case, from the case to a finding, and from the finding to the component involved. Figure 9.1 shows the full schema.

5.4.4 Graph Explainability

The diagnostic interface will let a user inspect the graph path that supports a candidate explanation. The graph must not create undocumented causal claims merely because two records share vocabulary.

5.5 M04: AI Diagnostic Workspace

5.5.1 Purpose

This is the flagship module of Halite. It allows technicians to investigate faults through a conversational, evidence-grounded interface.

5.5.2 Diagnostic Session Workflow

1. Select an asset.
2. Describe the observed symptoms.
3. Extract structured symptom features.
4. Retrieve relevant history.
5. Identify candidate fault cases.
6. Rank investigative hypotheses.
7. Validate supporting citations.
8. Produce an explanatory answer.
9. Ask follow-up questions.
10. Save the diagnostic session.

5.5.3 Symptom-Based Chat

The technician enters a natural-language description such as the following:

> A warning light remains illuminated after the previous maintenance action. What historical faults could explain this?

The system may ask for the aircraft or asset identity, the location of the symptom, the error or fault indication, whether the issue is intermittent, recent maintenance actions and operating conditions.

5.5.4 Diagnostic Hypotheses

The response may contain several hypotheses. Each hypothesis will include the candidate component or cause, the supporting historical evidence, the evidence strength, known limitations, relevant historical corrections and a suggested authorised investigation.

5.5.5 Follow-Up Reasoning

Clarification questions will be selected according to expected diagnostic information gain rather than generated arbitrarily.

5.5.6 Session Memory

Conversation state will preserve the selected asset, reported symptoms, clarification responses, retrieved record IDs, candidate hypotheses and user feedback.

5.5.7 Safety Policy

The assistant must not claim to certify a repair or independently determine airworthiness. Recommendations are framed as investigative decision support, and approved maintenance documentation and qualified personnel remain authoritative.

5.6 M05: Conflict Intelligence

5.6.1 Purpose

The conflict engine analyses inconsistencies within historical records.

5.6.2 Major Features

1. New-record comparison and relevant-prior retrieval.
2. Supersession classification.
3. Contradiction detection.
4. Partial-supersession handling.
5. Scope-disjoint classification.
6. Confidence assessment.
7. Supervisor review queue.
8. Audit trail.

9. Historical graph updates.

5.6.3 Record Statuses

Each record carries one of the following statuses: unscored, auto-accepted, needs review, accepted, rejected or superseded.

5.6.4 Human Review

A supervisor sees the current and previous records side by side. The review interface displays record identifiers, evidence excerpts, the predicted conflict class, the confidence value, the rationale and the action history.

5.6.5 Decision Rules

Unresolved contradictions must not be silently converted into confirmed supersession relationships. Human decisions are recorded with timestamps and reviewer identifiers.

5.7 M06: Maintenance Report Intelligence

5.7.1 Purpose

This module converts semi-structured reports into structured information.

5.7.2 Input Types

1. Structured CSV records.
2. Typed free-text reports.
3. Uploaded PDF reports.
4. Scanned documents.
5. Photographed typed reports.

OCR for scanned and photographed reports will be evaluated before it is treated as reliable.

5.7.3 Extracted Information

The extraction subsystem identifies the asset identifier, report identifier, report date, symptom description, component, component location, system code, inspection findings, maintenance action and corrective result.

5.7.4 Validation

Missing or ambiguous identifiers are flagged rather than silently fabricated, and every record preserves its original text.

5.8 M07: Technician Academy

5.8.1 Purpose

The training module transforms historical cases into diagnostic learning exercises.

5.8.2 Training Modes

- **Guided mode:** the learner receives incremental hints.
- **Assessment mode:** the learner investigates without immediate hints.
- **Case review mode:** the learner studies previous diagnostic mistakes and corrections.

5.8.3 Major Features

1. Historical case selection and generated case presentation.
2. Symptom-based questioning with simulated technician responses.
3. Diagnostic hypothesis submission.
4. Evidence inspection.
5. Hint progression.
6. Case completion feedback.
7. Training history and skill-gap analytics.

5.8.4 Scenario Creation

Training scenarios are derived from documented historical cases. Any generated detail that is not supported by the source record is labelled as synthetic.

5.8.5 Assessment Dimensions

The system evaluates relevant evidence selection, the quality of follow-up questions, recognition of uncertainty, identification of historical corrections and the final diagnostic justification.

5.8.6 Training Limitations

Historical SDR records may not contain enough information to establish the true root cause of every reported fault. Only suitably verified cases are used for ground-truth scoring. Other cases support open-ended exercises without a single claimed correct answer.

5.9 M08: Maintenance Analytics

5.9.1 Purpose

This module exposes patterns in recorded maintenance activity.

5.9.2 Major Features

1. Fault frequency trends.
2. Component issue distribution.
3. Repeat intervention patterns.
4. Common symptom categories.
5. Historical correction counts.
6. Diagnostic session and review queue trends.
7. Training performance trends.
8. Date-range filters and exportable reports.

5.9.3 Risk Indicators

A relative risk indicator may combine recorded fault recurrence, unresolved reviews and recent maintenance events. Such indicators are descriptive and are not labelled as validated failure probabilities without appropriate ground truth.

5.9.4 Forecasting Boundary

Parts-demand forecasting is excluded from the committed scope. FAA SDR data represents reported difficulties rather than complete parts consumption histories, so forecasting inventory demand from SDRs alone would be methodologically weak.

5.10 M09: Administration and Governance

5.10.1 Features

1. User and role management.
2. Organisation settings.
3. Data import management.
4. Dataset and model version tracking.
5. API health monitoring.
6. Review audit logs.
7. Diagnostic history governance.
8. Security configuration.
9. Data export.
10. Backup and recovery procedures.

6 Software Requirements Specification

6.1 Requirement Identification

Functional requirements use the prefix FR and non-functional requirements use the prefix NFR. Each requirement will be verified by at least one test or review procedure.

6.2 List of Features

The features of the system are grouped by module in Chapter 5. In summary, the platform provides authentication and tenancy, asset management, record ingestion, a maintenance knowledge graph, AI-assisted diagnostics, conflict intelligence with human review, technician training, analytics and administration.

6.3 Functional Requirements

Table 6.1: Functional Requirements

ID	Requirement
Authentication and Tenancy	
FR-01	The system shall authenticate authorised users.
FR-02	The system shall associate every user with one or more authorised organisations.
FR-03	The system shall enforce role-based permissions.
FR-04	The system shall prevent cross-tenant access.
Asset Management	
FR-05	The system shall allow authorised users to register maintenance assets.
FR-06	The system shall display chronological maintenance records for an asset.
FR-07	The system shall support asset search by identifier.
FR-08	The system shall display record-linked diagnostic and conflict information.
Record Processing	
FR-09	The system shall accept supported report formats.
FR-10	The system shall extract available asset, component, symptom and date information.
FR-11	The system shall retain the original source record.
FR-12	The system shall flag ambiguous extraction fields.
Knowledge Graph	
FR-13	The system shall create relationships among maintenance entities.
FR-14	The system shall allow authorised traversal of related historical cases.
FR-15	The system shall expose source provenance for graph-derived results.
Diagnostics	
FR-16	The system shall accept natural-language symptoms.
FR-17	The system shall retrieve relevant historical evidence.
FR-18	The system shall produce ranked diagnostic hypotheses when sufficient evidence exists.

ID	Requirement
FR-19	The system shall display supporting record references.
FR-20	The system shall request clarification when important information is missing.
FR-21	The system shall distinguish unsupported hypotheses from verified findings.
FR-22	The system shall preserve diagnostic session history.
Conflict Intelligence	
FR-23	The system shall compare new records with relevant prior records.
FR-24	The system shall classify potential historical conflicts.
FR-25	The system shall route uncertain cases for review.
FR-26	The system shall record supervisor review decisions.
FR-27	The system shall preserve an audit trail of record-status changes.
Training	
FR-28	The system shall allow a learner to start a diagnostic training case.
FR-29	The system shall record learner investigation steps.
FR-30	The system shall provide feedback linked to case evidence.
FR-31	The system shall store training completion history.
Analytics	
FR-32	The system shall display recorded fault trends.
FR-33	The system shall identify repeated recorded faults on the same asset.
FR-34	The system shall provide filters by asset, component and date range.
FR-35	The system shall distinguish descriptive statistics from predictive claims.

6.4 Non-Functional Requirements

All numerical targets in Table 6.2 are acceptance criteria to be verified during testing.

Table 6.2: Non-Functional Requirements

ID	Area	Acceptance Target
NFR-01	Availability	Deployed demonstration remains accessible during evaluation.
NFR-02	API performance	Non-generation requests meet p95 of at most 500 ms under the agreed test load.
NFR-03	Diagnostic latency	Median and p95 latency are measured under a controlled workload; the target is set after the baseline measurement.
NFR-04	Test quality	At least 80% coverage of selected core backend modules.
NFR-05	Security	Zero unauthorised tenant accesses in the security test suite.
NFR-06	Reliability	Failed external API operations are reported and retried safely.
NFR-07	Explainability	All displayed factual record citations resolve to real record IDs.
NFR-08	Usability	Task completion and System Usability Scale scores are measured during UAT.
NFR-09	Reproducibility	An independent setup succeeds from the documented repository.
NFR-10	Auditability	All review status changes are traceable to actor and time.

7 Use Cases and User Interface

7.1 Use Case Overview

Figure 7.1 shows the primary actors and use cases, and Table 7.1 lists the use cases with their actors.

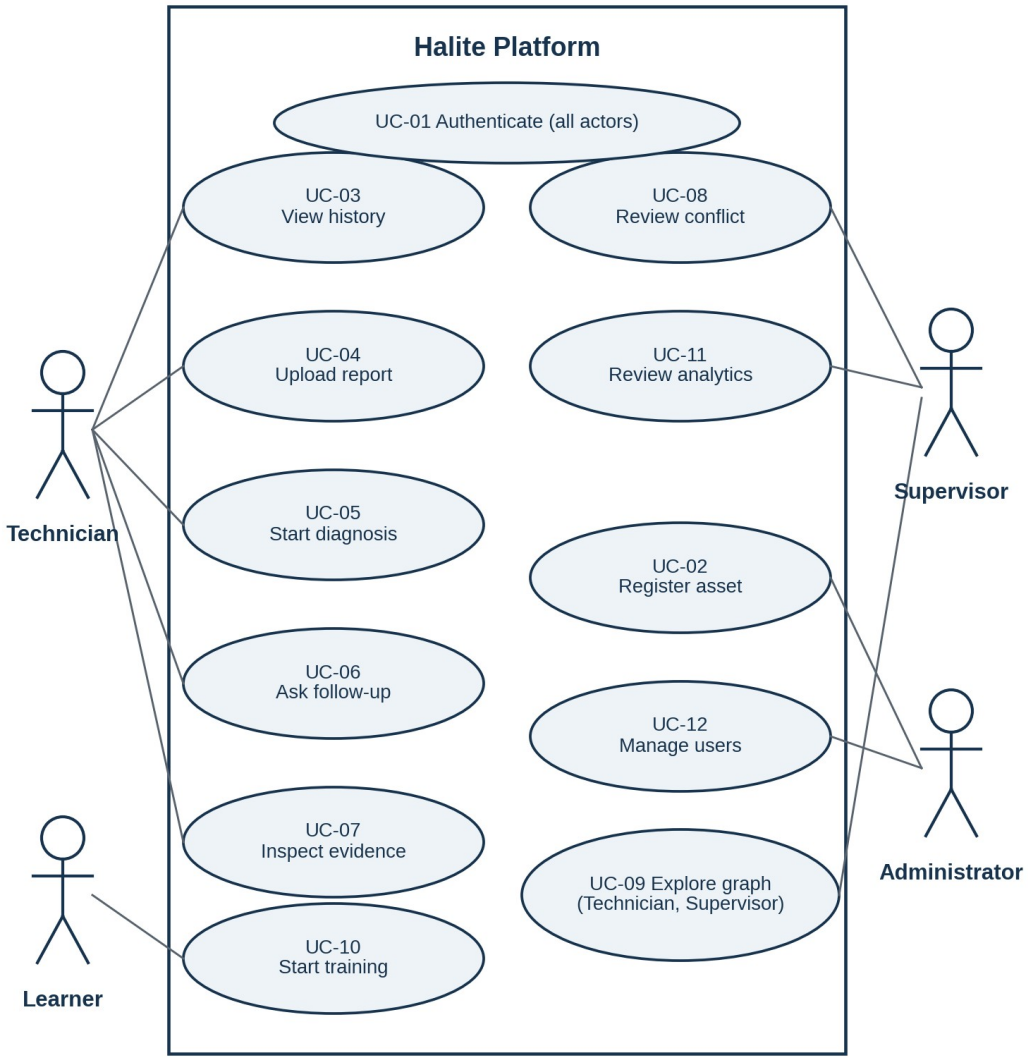


Figure 7.1: Primary Use Case Diagram

Table 7.1: Primary Use Cases

ID	Use Case	Primary Actor
UC-01	Authenticate	All users
UC-02	Register asset	Administrator
UC-03	View history	Technician
UC-04	Upload report	Technician
UC-05	Start diagnosis	Technician
UC-06	Ask follow-up	Technician

ID	Use Case	Primary Actor
UC-07	Inspect evidence	Technician
UC-08	Review conflict	Supervisor
UC-09	Explore graph	Authorised user
UC-10	Start training	Learner
UC-11	Review analytics	Supervisor
UC-12	Manage users	Administrator

7.2 UC-05: Start Diagnostic Investigation

Primary actor: Maintenance Technician.

Preconditions: the user is authenticated, an asset is selected and relevant historical data is available.

Main flow:

1. The technician opens Diagnostics and selects an asset.
2. The technician enters the observed symptoms.
3. The backend validates the input.
4. The symptom extractor produces structured information.
5. The retrieval engine obtains relevant historical cases.
6. The ranking engine scores candidate hypotheses.
7. The evidence validator checks record references.
8. The AI produces a grounded explanation.
9. The interface displays hypotheses and uncertainty.
10. The session is stored.

Alternative flow A (no relevant historical evidence): the system explicitly reports insufficient historical support and may request additional information.

Alternative flow B (external model unavailable): the system returns a controlled error or, where supported, a deterministic evidence summary.

Postcondition: a diagnostic session exists with its inputs, retrieval references and result metadata.

7.3 UC-08: Review Historical Conflict

Primary actor: Maintenance Supervisor.

Main flow:

1. The supervisor opens the review queue and selects a flagged case.
2. The supervisor inspects both records and the evidence.

3. The supervisor reviews the classification.
4. The supervisor accepts, rejects or edits the classification and confirms the decision.
5. The system updates the record status and writes an audit event.

Exception: unauthorised review requests are denied.

7.4 UC-10: Complete Training Scenario

Primary actor: Technician or Learner.

Main flow:

1. The learner selects a training category and starts a historical case.
2. The learner reads the symptom information.
3. The learner asks questions or inspects clues.
4. The learner submits a hypothesis and explains the supporting evidence.
5. The system provides case-specific feedback and saves the assessment record.

Postcondition: the system records learner performance.

7.5 Wireframes

Figures 7.2 and 7.3 show the planned layout of the two most frequently used screens: the diagnostic workspace and the asset detail page.

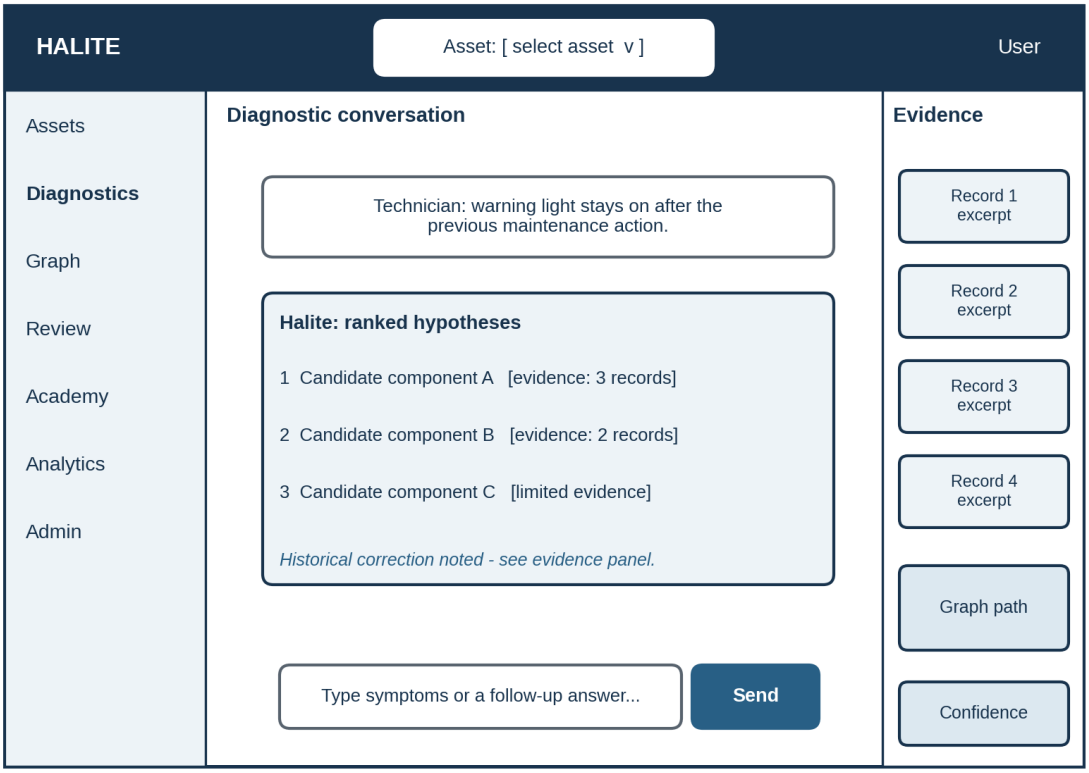


Figure 7.2: Wireframe of the Diagnostic Workspace

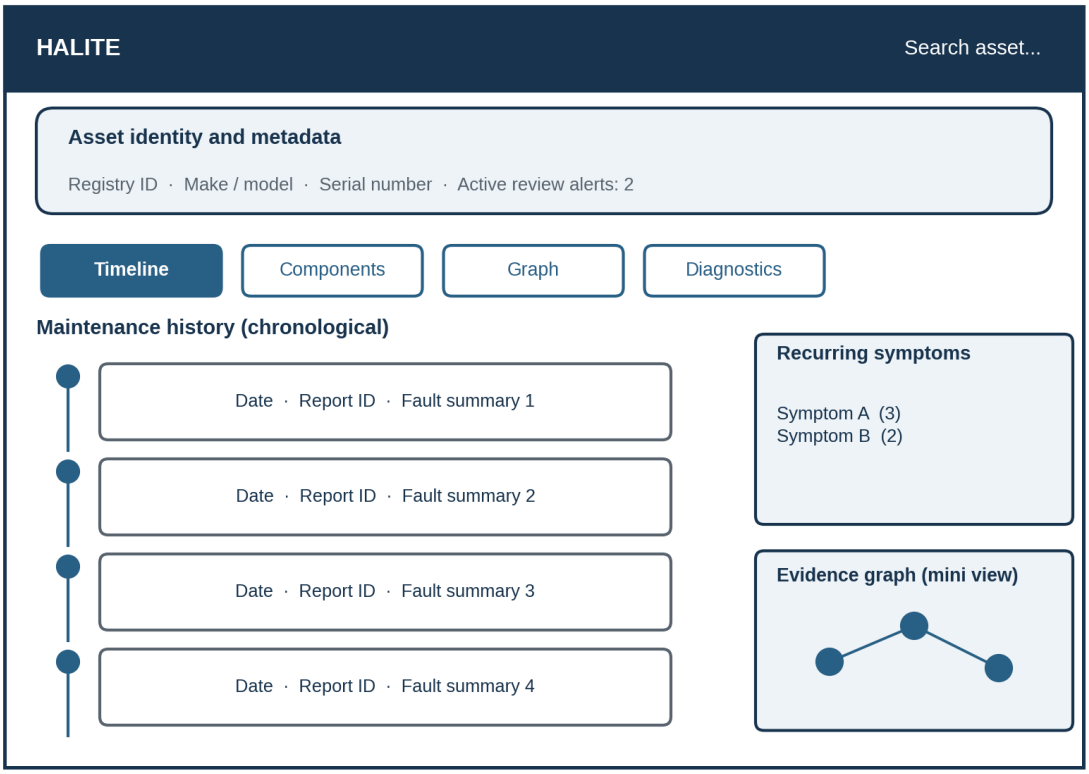


Figure 7.3: Wireframe of the Asset Detail Page

8 System Architecture

8.1 Architectural Style

Halite uses a web-based, modular client-server architecture organised into six layers: the presentation layer, the application API, the domain service layer, the diagnostic intelligence layer, the data persistence layer and the external AI service layer. Figure 8.1 shows the high-level structure.

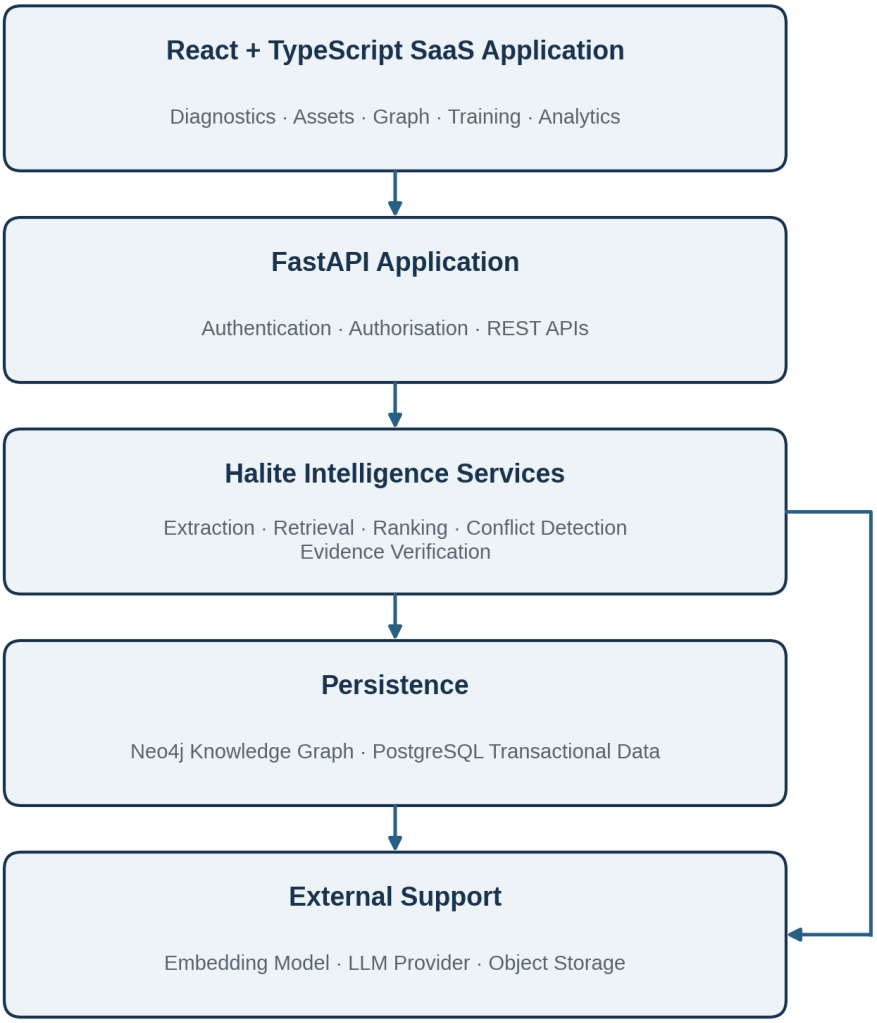


Figure 8.1: High-Level System Architecture

8.2 Frontend

The frontend is a React application written in TypeScript. It is responsible for user interaction, application navigation, the diagnostic chat, asset dashboards, graph visualisation, review workflows, training interfaces and analytics dashboards.

8.3 Backend

The backend is built with FastAPI. It is responsible for API routing, authentication, authorisation, data validation, diagnostic orchestration, conflict workflow coordination, external AI integrations and audit event creation.

8.4 Graph Database

Neo4j stores the maintenance knowledge graph because the core queries are relationship traversals. Its responsibilities include asset relationships, historical case associations, component links, symptom relationships, record chronology, supersession edges and evidence traversals.

8.5 Relational Database

PostgreSQL stores transactional SaaS data: organisations, users, memberships, diagnostic sessions, training attempts, notifications and audit events. Separating transactional data from the graph keeps tenant administration and auditing on a mature relational store while the graph database remains focused on retrieval.

8.6 External Language Models

A hosted large language model supports symptom interpretation, clarification questions, explanation generation, training dialogue and record classification assistance. The model is never permitted to create factual record IDs. Every generated citation is checked against the authorised retrieval results before it is displayed.

8.7 Deployment Architecture

The deployment uses containerised backend services and hosted frontend delivery. It comprises frontend hosting, a backend container, the graph database, the relational database, secret management, CI/CD workflows and error monitoring. Specific hosting targets will be selected on the basis of cost, availability and resource requirements.

9 Data and Knowledge Graph Design

9.1 Primary Data Source

The research and demonstration domain is the FAA Service Difficulty Report (SDR) collection. The relevant fields include DifficultyDate, RegistryNNumber, aircraft make and model, aircraft serial number, PartName, PartCondition, PartLocation, JASCCode and Discrepancy. These fields provide the foundation for asset-specific history retrieval.

9.2 Evaluation Corpus

The initial evaluation corpus is described in Table 9.1. All figures will be reproduced from the evaluation data and documented in the dataset register.

Table 9.1: Planned Evaluation Corpus

Property	Planned Value
Maintenance service records	2,079
Approximate airframes	60
Author-recorded correction pairs	141
Candidate-level conflict evaluation examples	350
Conflict baseline	Recency-based comparison

The 141 author-recorded correction pairs will be analysed to measure how often a correcting record names a different part, a different JASC code or a different part location. These proportions describe correction pairs only and will not be generalised to all maintenance diagnoses. Halite will use observed case relationships and will not generate general component-dependency edges unless statistical support is demonstrated against a predefined support threshold.

9.3 Graph Node Types

Table 9.2: Knowledge Graph Schema: Node Types

Node	Purpose
Asset	Physical maintenance asset
Component	Technical component
ServiceRecord	Original maintenance report
Symptom	Reported fault observation
FaultCase	Grouped historical incident
Inspection	Recorded inspection action
Finding	Observed technical finding
RepairAction	Maintenance intervention

Node	Purpose
SourceDocument	Original evidence source
Adjudication	Conflict review decision

9.4 Graph Edge Types

Table 9.3: Knowledge Graph Schema: Relationship Types

Relationship	Meaning
HAS_RECORD	Asset to service record
REPORTS	Record to symptom
INVOLVES	Case to component
DESCRIBES	Record to finding
ACTION_ON	Repair to component
EXTRACTED_FROM	Entity to source
PRECEDED_BY	Chronological relation
SUPERSEDES	Later record replaces earlier
CONTRADICTS	Conflicting record pair
FLAGGED_BY	Record to adjudication
RESOLVED_BY	Case to verified action

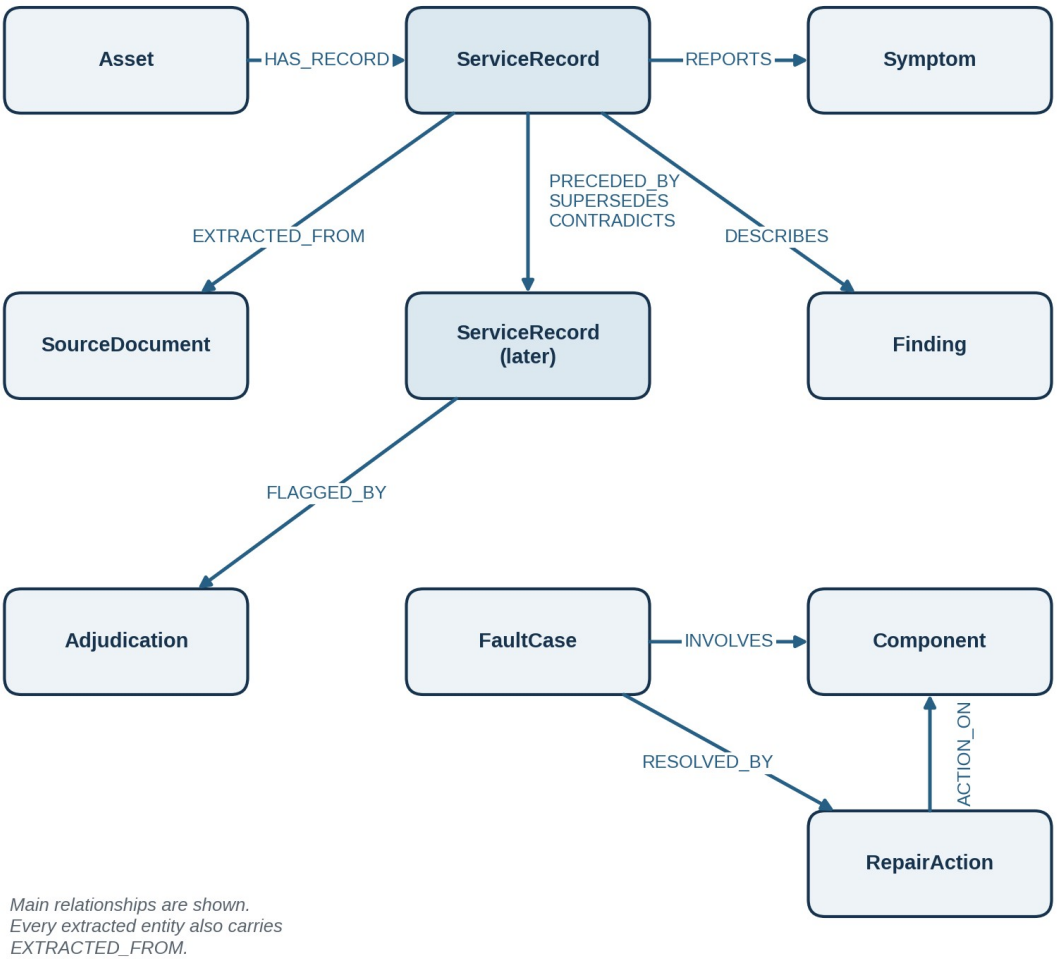


Figure 9.1: Knowledge Graph Schema

9.5 Graph Integrity Rules

1. Every service record retains its source ID.
2. Every asset relationship uses a normalised asset identifier.
3. No fabricated historical record may appear as a real source.
4. Contradiction does not automatically imply supersession.
5. Every adjudication preserves its decision metadata.
6. Organisation access rules apply to every query.
7. Unverified component relationships are not presented as proven causal dependencies.

9.6 Fault Case Model

A historical fault case is modelled as a tuple:

$$C_i = (A_i,\ S_i,\ P_i,\ F_i,\ R_i,\ T_i) \tag{1}$$

where A_i is the asset identity, S_i the symptoms, P_i the components, F_i the findings, R_i the repairs and T_i the temporal context. Not every record contains every attribute, so missing information is represented explicitly.

9.7 Entity Resolution

Maintenance reports may use variant labels for the same asset or component. The resolver considers exact identifiers, normalised identifiers, domain codes, component location and contextual similarity. An uncertain match is retained as provisional rather than merged.

9.8 Data Provenance

Every extracted graph fact stores a reference to the original record and the extraction method. The system differentiates four kinds of information: original observations, extracted facts, model-inferred relations and human-approved relations.

10 Diagnostic Intelligence Algorithms

10.1 Diagnostic Processing Pipeline

The diagnostic pipeline contains the eight stages shown in Figure 10.1: input validation, symptom extraction, asset-specific retrieval, graph candidate expansion, semantic evidence retrieval, hypothesis ranking, evidence validation, and explanation and interaction.

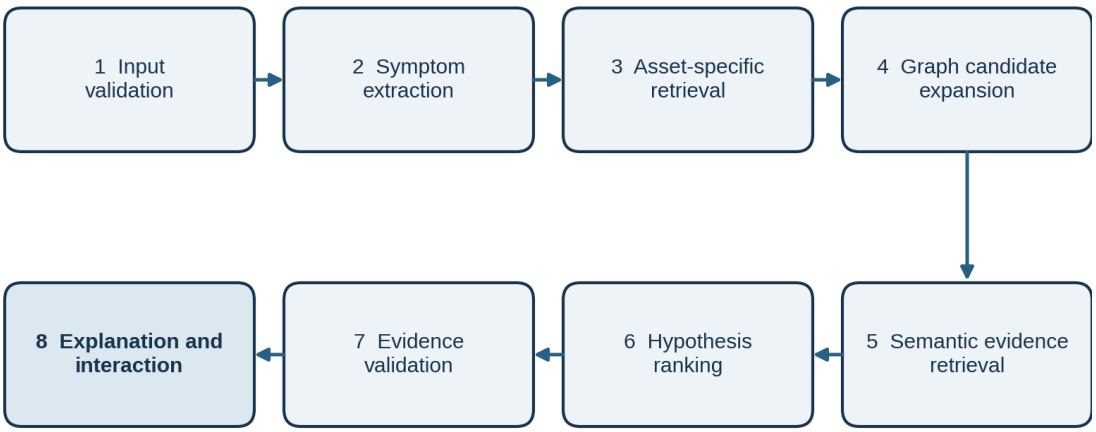


Figure 10.1: Diagnostic Processing Pipeline

10.2 Symptom Extraction

Given an input text q, the extractor produces a structured description:

$$E(q) = \{\text{symptoms},\ \text{components},\ \text{conditions},\ \text{locations}\} \tag{2}$$

Extraction is probabilistic when a language model is used, so critical identifiers are checked against known asset records.

10.3 Graph-Constrained Retrieval

Let $G = (V, E)$ be the maintenance graph. For a selected asset a, candidate records are restricted according to explicit graph relationships:

$$R_a = \{\, r \mid (a, r) \in E_{\text{HAS_RECORD}} \,\} \tag{3}$$

Additional candidates may be discovered through related component or symptom connections. Cross-asset evidence is always labelled as such.

10.4 Semantic Retrieval

Each record is represented by an embedding vector, and similarity between a query q and a record r is computed using cosine similarity:

$$\mathrm{sim}(q, r) = \frac{\mathbf{e}_q \cdot \mathbf{e}_r}{\|\mathbf{e}_q\| \, \|\mathbf{e}_r\|} \tag{4}$$

The query returns the highest-scoring records from the authorised candidate set.

10.5 Hybrid Candidate Retrieval

The hybrid method combines graph and semantic evidence in a weighted ranking expression:

$$S(r, q) = w_1 S_{\text{sem}} + w_2 S_{\text{asset}} + w_3 S_{\text{comp}} + w_4 S_{\text{temp}} + w_5 S_{\text{evid}} \tag{5}$$

$$\sum_{i=1}^{5} w_i = 1 \tag{6}$$

The weights are not assumed to be optimal. They are selected using validation data and then evaluated on a held-out test set.

10.6 Historical Correction Penalty

When a retrieved record is marked as superseded, the ranking system may reduce its diagnostic authority. The record can nevertheless remain useful as historical evidence of an unsuccessful intervention. The adjusted score is:

$$S'(r, q) = S(r, q) - \lambda \, C(r) \tag{7}$$

where $C(r)$ reflects the adjudicated correction status. A disputed record is not treated as definitively false without sufficient evidence.

10.7 Fault Hypothesis Construction

Candidate evidence is grouped by fault hypothesis. For a hypothesis h, the support score is:

$$H(h) = \alpha E_h + \beta G_h + \gamma V_h - \delta U_h \tag{8}$$

where E_h is the textual evidence relevance, G_h the graph relationship support, V_h the verified case support and U_h the contradiction or uncertainty penalty. This formula is a design hypothesis that the evaluation in Chapter 11 will test; it is not assumed to be an optimal model.

10.8 Confidence and Abstention

The system distinguishes a ranking score from a calibrated probability of correctness. A raw language-model confidence value is not treated as sufficient evidence of reliability, and calibration is evaluated against labelled cases. When evidence is weak, the system will:

1. Ask for clarification.
2. Display uncertainty.
3. Return a limited evidence summary.
4. Recommend human review.
5. Abstain from unsupported conclusions.

10.9 Question Selection

The diagnostic chat may select follow-up questions by their expected ability to distinguish hypotheses. For a candidate question x, the expected information gain is:

$$IG(x) = H(F) - \mathbb{E}\left[H(F \mid x) \right] \tag{9}$$

where F is the current hypothesis distribution. This information-gain approach is optional until a sufficiently reliable hypothesis model is established.

10.10 Evidence Verification

Before presenting an explanation, the system performs the following steps:

1. Extract the cited record IDs.
2. Check each against the retrieved authorised records.
3. Remove unsupported references.
4. Check the claim against the cited excerpt.
5. Record the validation results.

No invented citation is presented as a verified record.

10.11 Algorithm Pseudocode

```
FUNCTION Diagnose(asset, symptoms, user):
    VerifyAccess(user, asset)
    structured = ExtractSymptoms(symptoms)
    history = GetAssetHistory(asset)
    graphCandidates = GraphRetrieve(asset, structured)
    semanticCandidates = SemanticRetrieve(structured, history)
    candidates = MergeAndDeduplicate(graphCandidates,
                                     semanticCandidates)
    evidence = ValidateRecordSources(candidates)
    evidence = ApplyCorrectionAwareness(evidence)
    hypotheses = BuildFaultHypotheses(structured, evidence)
    ranked = RankHypotheses(hypotheses)
    IF EvidenceInsufficient(ranked):
        RETURN AskClarifyingQuestion()
    answer = GenerateGroundedExplanation(ranked, evidence)
    answer = VerifyCitations(answer)
    SaveDiagnosticSession(user, asset, symptoms,
                          ranked, answer)
    RETURN answer
END FUNCTION
```

10.12 Conflict Classification Engine

For every new record, the conflict engine retrieves candidate prior records for the same asset, compares them and assigns one of five classes: contradicts, supersedes, partially supersedes, scope disjoint or no conflict. A confidence value accompanies each decision. Low-confidence decisions are routed to the supervisor review queue and are never converted silently into confirmed supersession edges.

10.13 Diagnostic Evaluation Baselines

The proposed method will be compared with keyword-only retrieval, semantic retrieval, graph-only retrieval, hybrid retrieval and a direct language-model response without external evidence. This comparison is central to demonstrating that the graph and ranking logic provide measurable value beyond a chatbot.

11 Evaluation and Research Methodology

11.1 Research Design

The evaluation uses controlled, reproducible experiments. Its primary purpose is to determine whether the proposed intelligence layer improves measurable diagnostic outcomes.

11.2 Dataset Preparation

The dataset pipeline consists of the following steps:

1. Data acquisition.
2. Licence verification.
3. Deduplication.
4. Identifier normalisation.
5. Temporal ordering.
6. Record linking.
7. Evidence labelling.
8. Train, validation and test splits.

11.3 Avoiding Data Leakage

Related records from the same fault episode can create leakage between training and evaluation sets. Where feasible, splits will group related incidents. A separate temporal holdout may be used to test prediction on later records. Hyperparameters will not be tuned on the final test data.

11.4 Diagnostic Ground Truth

Not every service report establishes a confirmed cause. The evaluation therefore distinguishes four categories: author-recorded corrections, explicitly confirmed findings, probable but unconfirmed diagnoses and unresolved reports. Only appropriate cases receive single-answer diagnostic labels.

11.5 Retrieval Metrics

Retrieval quality is measured with precision at k, recall at k and mean reciprocal rank:

$$P@k = \frac{\text{Relevant records in top } k}{k} \tag{10}$$

$$R@k = \frac{\text{Relevant records in top } k}{\text{Total relevant records}} \tag{11}$$

$$\mathrm{MRR} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\mathrm{rank}_i} \tag{12}$$

11.6 Diagnostic Ranking Metrics

- **Top-*k* accuracy:** whether an accepted diagnostic label appears within the top *k* hypotheses.
- **Mean rank:** the mean rank of the correct hypotheses, as an additional ranking-quality measure.
- **Coverage:** the proportion of cases for which the system provides a substantive diagnostic response.

11.7 Confidence Evaluation

The evaluation reports calibration error, reliability plots, selective accuracy, coverage versus error and the unsupported suggestion rate. The relationship between confidence and actual correctness is measured, not assumed.

11.8 Citation Evaluation

Citation validity is measured by checking whether references resolve to actual authorised records. Additional human review assesses whether the cited content supports the associated diagnostic claim.

11.9 Conflict Evaluation

The conflict detection engine is evaluated at the pair level and the candidate level against a recency-based baseline. The reported measures are precision, recall, F1 score, false-conflict rate, review queue size, automation coverage and a qualitative error analysis.

11.10 Ablation Study

The team will compare performance when individual components are removed. The variants are the full diagnostic pipeline, a version without graph constraints, a version without correction awareness, a version without evidence reranking and a version without uncertainty gating. This identifies which components contribute meaningfully to performance.

11.11 User Acceptance Testing

User acceptance testing evaluates realistic technician and supervisor tasks. The measured factors are task completion rate, completion time, evidence interpretation accuracy, perceived usability, trust calibration and the ability to identify unsupported conclusions. If qualified maintenance

personnel cannot be recruited, the team will declare proxy participants. Proxy evaluation cannot justify claims of real-world diagnostic competence.

11.12 Training Evaluation

Where feasible, two comparable learner groups will complete historical diagnostic exercises. One group receives ordinary reference materials and the other uses Halite's guided training interface. Measurements include diagnostic task accuracy, evidence selection accuracy, completion time and retention on new cases. Any educational improvement claim requires actual experimental results.

12 Security and Safety

12.1 Security Requirements

The platform must protect organisational maintenance data. The required controls are authentication, authorisation, tenant isolation, input validation, secret management, audit logging, secure transport and controlled error reporting.

12.2 Multi-Tenant Isolation

Every organisation owns an isolated logical dataset. Queries include the appropriate tenant constraints, and an authenticated user cannot access another organisation's private records.

12.3 LLM Security

Maintenance documents are treated as untrusted input. Text embedded inside uploaded reports must not override system authorisation policies. The application will be tested for prompt injection, citation fabrication, sensitive data leakage, unauthorised graph retrieval and unsafe tool invocation.

12.4 Operational Safety

Halite is an investigative decision-support system. Uncertain hypotheses are never presented as verified diagnoses, and the interface repeatedly states that approved maintenance documentation and qualified personnel remain authoritative. The system never approves repairs, certifies airworthiness or authorises maintenance release.

13 Iteration Plan

13.1 Development Approach

Halite follows an iterative, incremental development strategy consistent with the FYP handbook. Every iteration will produce demonstrable software together with the supporting analysis, design, test and deployment artefacts. Figure 13.1 gives an overview of the plan, and Table 13.1 summarises the focus of each evaluation stage.

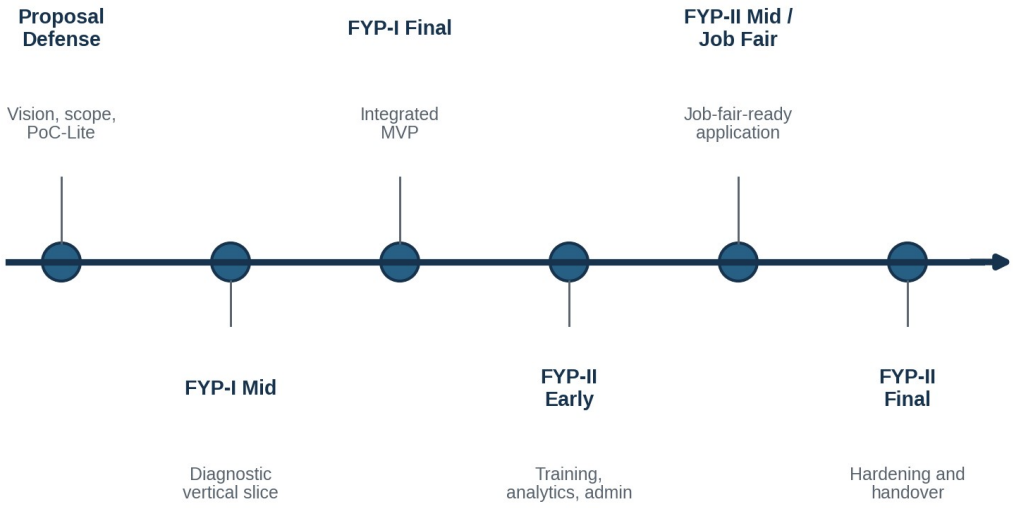


Figure 13.1: Iteration Roadmap

Table 13.1: Summary of Evaluation Stages

Stage	Focus
Proposal Defense	Vision, research gap, requirements, architecture and PoC-Lite.
FYP-I Mid	Working maintenance diagnostic vertical slice on real data.
FYP-I Final	Integrated maintenance intelligence minimum viable product.
FYP-II Early	Extension into a complete maintenance SaaS product.
FYP-II Mid / Job Fair	Job-fair-ready integrated application.
FYP-II Final	Hardening, final evaluation, documentation and handover.

13.2 Proposal Defense

At the proposal defense, the team will present the project vision, problem statement, research gap, requirements, architecture and development plan, together with a PoC-Lite. The PoC-Lite will demonstrate maintenance record ingestion, asset history timelines, graph-based candidate selection, conflict classification, a human review queue and a diagnostic query interface that returns evidence-linked results.

13.3 Iteration 1: FYP-I Mid Evaluation

13.3.1 Objective

Demonstrate a working maintenance diagnostic vertical slice supported by real historical data.

13.3.2 Target Features

1. Maintenance-focused user interface.
2. Asset history browser.
3. Live knowledge graph.
4. Symptom-based diagnostic input.
5. Historical evidence retrieval.
6. Ranked investigation suggestions.
7. Conflict review capability.
8. API integration.
9. Automated tests.
10. GitHub Actions CI/CD.

13.3.3 Demonstration Flow

1. Open an asset.
2. View the maintenance history.
3. Enter a fault symptom.
4. Retrieve related records.
5. Generate an evidence-backed response.
6. Display real source references.
7. Show a relevant historical correction.
8. Demonstrate CI results.

13.3.4 Documentation

Project vision, problem and research gap, functional and non-functional requirements, use cases, architecture, data model, initial implementation, test cases, iteration plan, team ownership and tool disclosure.

13.4 Iteration 2: FYP-I Final Evaluation

13.4.1 Objective

Deliver an integrated maintenance intelligence minimum viable product.

13.4.2 Target Features

1. Improved diagnostic ranking.
2. Hybrid retrieval.
3. Multi-turn diagnostic sessions.
4. Asset and component graph.
5. Record ingestion.
6. Role-based access.
7. Review queue.
8. Historical correction handling.
9. Session history.
10. Deployed demonstration.

13.4.3 Academic Deliverables

Updated SRS, software design specification (SDS), functional system, unit tests, integration tests, initial algorithm evaluation, repository evidence and an updated risk register.

13.5 Iteration 3: FYP-II Early Development

13.5.1 Objective

Extend the integrated minimum viable product into a complete maintenance SaaS product.

13.5.2 Target Features

1. Advanced clarification engine.
2. Improved diagnostic ranking.
3. Technician Academy and training assessments.
4. Analytics dashboards and recurring-fault insights.
5. Supervisor collaboration.
6. Organisation administration.
7. Graph exploration.
8. Expanded test coverage.

13.6 Iteration 4: FYP-II Mid and Job Fair

13.6.1 Objective

Present a job-fair-ready, integrated application. The internal target is approximately 90 to 95 percent of the planned functional scope.

13.6.2 Expected Demonstration

1. Organisation login.
2. Asset selection.
3. Diagnostic chat.
4. Evidence-based ranking.
5. Knowledge graph path.
6. Historical correction.
7. Supervisor review.
8. Training scenario.
9. Maintenance analytics.
10. Deployment and CI/CD evidence.

13.7 Iteration 5: FYP-II Final

The final phase prioritises defect correction, final evaluation, security review, performance testing, documentation completion, installation instructions, the final technical defence and handover. No major new modules will be introduced after FYP-II Mid without strong justification.

14 Team Responsibilities

14.1 Team Composition

Table 14.1 lists the primary ownership area of each team member.

Table 14.1: Team Ownership

Member	Primary Ownership
Shahmeer Atif	Diagnostic interaction, frontend integration, evidence presentation and product UX.
Muhammad Umar	Knowledge graph, retrieval algorithms, fault ranking and technical evaluation.
Muhammad Usman Al Haq	Data processing, conflict intelligence, CI/CD, backend integration and deployment.

14.2 Shahmeer Atif

1. Diagnostic workspace UX.
2. Multi-turn chat interaction.
3. Asset interface integration.
4. Evidence visualisation.
5. Graph interaction UX.
6. Frontend API integration.
7. End-to-end workflow testing.

14.3 Muhammad Umar

1. Graph schema.
2. Entity resolution.
3. Graph traversal.
4. Semantic retrieval.
5. Hybrid ranking.
6. Diagnostic scoring.
7. Ablation evaluation.

14.4 Muhammad Usman Al Haq

1. Dataset ingestion.
2. Source normalisation.
3. Conflict detection.
4. Review workflow backend.

5. Automated testing setup.
6. GitHub Actions.
7. Deployment.
8. Operational monitoring.

14.5 Shared Responsibilities

All team members participate in architecture decisions, requirements review, code reviews, integration, test design, academic documentation, supervisor meetings and demonstration preparation.

14.6 Individual Ownership Evidence

Every member maintains a traceable record of assigned tasks, implemented features, pull requests, test contributions, technical decisions, evaluation work and demonstrated knowledge. Commit volume alone is not treated as technical contribution.

15 CI/CD and Software Engineering Practices

15.1 Version Control

GitHub is the primary source repository. The workflow uses a main branch, a development branch where appropriate, feature branches, pull requests, code review and tagged releases.

15.2 Branch Conventions

Example branches:

```
main
develop
feature/diagnostic-chat
feature/graph-retrieval
feature/conflict-review
feature/training-engine
feature/analytics
fix/citation-validation
```

15.3 Continuous Integration

Every pull request triggers dependency installation, backend linting, frontend linting, backend unit tests, a frontend build, integration checks and test artefact generation.

15.4 Continuous Deployment

A verified release workflow deploys approved changes to the demonstration environment. The deployment pipeline supports rollback or redeployment to a known working version.

15.5 Repository Structure

The planned repository layout is:

```
halite/
  frontend/
    src/
      components/  pages/  features/  services/
  backend/
    app/
      api/  auth/  diagnostics/  graph/  conflict/
      ingestion/  training/  analytics/
    tests/
  data/
    schemas/  evaluation/
  docs/
    srs/  sds/  diagrams/  research/
  .github/
    workflows/
  docker-compose.yml
  README.md
```

15.6 Continuous Integration Evidence

For evaluation, the repository will show successful workflow runs, automatically executed tests, meaningful pull requests, individual contributions, release history and reproducible installation.

15.7 Definition of Done

A feature is considered complete only when:

1. Requirements are defined.
2. Code is implemented.
3. Tests are present and pass.
4. Integration is verified.
5. Documentation is updated.
6. The feature is demonstrable.

16 Testing Strategy

16.1 Testing Objectives

Testing verifies functional correctness, diagnostic evidence validity, security boundaries, graph consistency, API behaviour, SaaS workflows and deployment reliability.

16.2 Unit Testing

Unit tests cover symptom normalisation, candidate filtering, graph query construction, ranking calculations, conflict decisions, citation validation and permission checks.

16.3 Integration Testing

Integration tests verify the following interfaces:

1. Frontend to API.
2. API to Neo4j.
3. API to the relational database.
4. Retrieval to ranking.
5. Ranking to explanation.
6. Conflict detection to the review queue.
7. Review action to the audit log.

16.4 System Testing

System-level testing uses realistic maintenance workflows and demonstrates both success and failure cases.

16.5 Security Testing

Security tests cover unauthorised routes, cross-tenant reads, malformed inputs, token expiry, prompt injection, citation manipulation and sensitive error leakage.

16.6 Performance Testing

Performance tests measure retrieval latency, diagnostic response latency, concurrent user behaviour, graph traversal time, API error rate and resource consumption.

16.7 Failure-Mode Testing

Important failure scenarios include:

1. Missing asset.
2. Empty maintenance history.
3. Invalid report ID.
4. Conflicting source evidence.
5. Language-model timeout.
6. Database disconnection.
7. Unauthorised review request.
8. Duplicate report submission.
9. Low-confidence diagnosis.

16.8 Acceptance Testing

The system is acceptable only when core workflows complete using real application data. A static interface or hardcoded response does not satisfy the live prototype requirement.

17 Project Risks and Mitigation

Table 17.1 lists the identified risks and their planned mitigation.

Table 17.1: Project Risk Register

ID	Risk	Mitigation
R01	Insufficient diagnostic ground truth	Separate confirmed findings from unresolved cases; restrict quantitative evaluation.
R02	Poor diagnostic ranking accuracy	Compare baselines; analyse errors; revise ranking features.
R03	Graph adds no measurable value	Conduct an ablation study; report negative results.
R04	Language-model hallucination	Validate citations; use grounded prompts; permit abstention.
R05	Historical corrections are ambiguous	Use explicit labels and human review.
R06	Training cases lack confirmed answers	Use verified cases for scoring; label synthetic scenarios.
R07	Stakeholder access unavailable	Recruit early; declare proxy limitations.
R08	External API cost or downtime	Caching, retries, resource limits and fallback behaviour.
R09	Integration delays	Contract-first APIs; frequent integration; small milestones.
R10	Scope expansion	Prioritise flagship modules; freeze major features before FYP-II Mid.
R11	Data licensing uncertainty	Maintain a dataset licence register.
R12	Tenant data leakage	Mandatory authorisation; security test suite.
R13	Incomplete CI/CD	Use automated test and deployment workflows from early development.
R14	Weak individual ownership	Track tasks, pull requests, tests and architecture decisions per member.
R15	Misleading safety claims	Position as decision support; require human authority.
R16	Evaluation leakage	Group related incidents and use held-out tests.

18 Expected Results and Deliverables

18.1 Expected Technical Results

The project is expected to produce an integrated SaaS system, a maintenance knowledge graph, an evidence-grounded diagnostic engine, a conflict-aware record workflow, a technician training module, maintenance analytics, automated test suites, reproducible evaluation results and deployment documentation.

18.2 Expected Research Results

The team will report the actual outcomes of retrieval comparisons, fault ranking evaluation, citation validation, conflict classification, confidence calibration, ablation experiments, performance testing and user acceptance testing. A negative result does not invalidate the project if the experiment is properly designed and interpreted.

18.3 Documentation Deliverables

1. Master proposal and final project report.
2. SRS and SDS, with architecture diagrams and data model.
3. Algorithm specifications.
4. Test plan and evaluation report.
5. User manual and deployment instructions.
6. Project risk register.
7. Contribution evidence and AI usage disclosure.

18.4 Commercial Demonstration

The product demonstration focuses on one continuous maintenance workflow:

> A technician describes a fault. Halite retrieves the asset's maintenance history, identifies related cases, explains possible investigation paths, exposes a historical correction and allows the supervisor to review the underlying evidence.

The training and analytics modules then demonstrate how the same knowledge foundation supports additional organisational value.

19 Conclusion

Halite is an integrated AI-assisted maintenance intelligence platform that places evidence-grounded diagnostics at its centre. The knowledge graph provides structured asset relationships. The diagnostic engine retrieves and ranks historical evidence. The conflict engine identifies potentially outdated findings. The training module transforms suitable cases into educational exercises, and the SaaS layer makes these capabilities available through coherent workflows.

The academic success of the project depends on demonstrating that its student-owned retrieval, ranking, conflict and uncertainty mechanisms provide measurable value beyond standard language model generation. The final system will be deployed, tested, documented, reproducible and defensible by all three team members.

References

Asai, A. et al. (2023) *Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection*, arXiv preprint.

Edge, D. et al. (2024) *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, arXiv preprint.

FAST School of Computing (2026) *BS Final Year Project Handbook, Third Edition (Draft)*, National University of Computer and Emerging Sciences, Islamabad.

Federal Aviation Administration (n.d.) *Service Difficulty Reporting Data and Documentation*, FAA, United States Department of Transportation.

Lewis, P. et al. (2020) “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, *Advances in Neural Information Processing Systems (NeurIPS)*.

Appendix A: Mid Evaluation Readiness Checklist

A.1 Live Core Module

1. Asset selection works.
2. Historical records load.
3. The diagnostic query executes.
4. Retrieval returns real records.
5. Citations resolve correctly.
6. A diagnostic response is generated.
7. Insufficient-evidence behaviour works.
8. A conflict example can be demonstrated.

A.2 CI/CD

1. The repository contains feature branches.
2. Pull requests are visible.
3. Unit tests run automatically.
4. The frontend and backend build automatically.
5. Deployment is reproducible.
6. The release commit is tagged.

A.3 Documentation

1. Problem statement and research gap documented.
2. SRS requirements numbered.
3. Use cases prepared.
4. Architecture justified.
5. Data schema available.
6. Test cases documented.
7. Individual contributions recorded.
8. Risks documented.
9. AI usage disclosed.

Appendix B: Diagnostic Acceptance Cases

B.1 Case A: Known Historical Correction

Input: a symptom matching a known historical correction sequence.

Expected behaviour:

1. Retrieve both records in temporal order.
2. Surface the corrective finding.
3. Mark the older intervention as historical.
4. Cite real record identifiers.

B.2 Case B: No Relevant History

Input: a symptom with no supporting historical cases.

Expected behaviour:

1. Avoid fabricated evidence.
2. State insufficient historical support.
3. Ask clarifying questions without false certainty.

B.3 Case C: Conflicting Evidence

Input: two reports with incompatible conclusions.

Expected behaviour:

1. Retrieve both reports with their source references.
2. Show the disagreement.
3. Avoid silently choosing a winner.
4. Escalate where appropriate.

B.4 Case D: Cross-Tenant Access

Input: a user requests another organisation's asset history.

Expected behaviour:

1. Deny access.
2. Return no private records.
3. Preserve appropriate security logs.

Appendix C: Research Experiment Matrix

Table C.1: Research Experiment Matrix

Experiment	Baseline	Main Measurements
Case retrieval	Keyword search	Precision@k, Recall@k, MRR.
Hybrid retrieval	Embedding search	Retrieval quality, latency.
Fault ranking	Direct LLM	Top-k accuracy, mean rank.
Correction awareness	No correction handling	Unsupported evidence rate.
Confidence gating	Always answer	Selective accuracy, coverage.
Conflict classification	Recency baseline	Precision, recall, F1, false conflicts.
Training evaluation	Traditional materials	Task accuracy, time, retention.
System performance	Local baseline	p50 and p95 latency, error rate.

Appendix D: Final Deliverable Checklist

Table D.1: Final Deliverable Checklist

Technical Deliverables	Documentation and Evidence
Deployed frontend	Software Requirements Specification
Deployed backend	Software Design Specification
Maintenance graph	API documentation
Diagnostic reasoning pipeline	Data documentation
Conflict review workflow	User manual
Maintenance ingestion	Deployment manual
Technician training	Evaluation benchmarks
Maintenance analytics	Ablation study
Authentication and RBAC	Usability evaluation
Organisation isolation	Security and performance testing
Automated tests	AI disclosure
CI/CD pipeline	Contribution log
Job fair demonstration	Final technical report
Backup demonstration recording	